Parallel Programming Lab Programs

CUDA and MPI programs with source code, compile/run commands, expected output, and explanations

Programs included

• 1. OpenMP Matrix Multiplication in C

• 2. CUDA Prime Number Finder (2 to 50)

• 3. CUDA Sum Reduction

• 4. MPI Parallel Sorting

Environment notes

OpenMP programs need a C compiler with OpenMP support. CUDA programs need the NVIDIA CUDA Toolkit, a compatible NVIDIA GPU, and a working driver. MPI programs need an MPI implementation such as Open MPI or MPICH. Commands below are typical Linux terminal commands.

1. OpenMP Matrix Multiplication

Purpose: Multiply two compatible matrices. OpenMP parallelizes the outer loop so separate result rows can be calculated by different threads.

Source file: **matrix_multiplication.c**

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    int r1, c1, r2, c2;
    int A[10][10], B[10][10], C[10][10];

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    if (r1 < 1 || r1 > 10 || c1 < 1 || c1 > 10 ||
        r2 < 1 || r2 > 10 || c2 < 1 || c2 > 10) {
        printf("Dimensions must be between 1 and 10.\\n");
        return 1;
    }

    if (c1 != r2) {
        printf("Matrix multiplication not possible!\\n");
        return 0;
    }

    printf("Enter elements of Matrix A:\\n");
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c1; j++)
            scanf("%d", &A[i][j]);

    printf("Enter elements of Matrix B:\\n");
    for (int i = 0; i < r2; i++)
        for (int j = 0; j < c2; j++)
            scanf("%d", &B[i][j]);

    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++)
            C[i][j] = 0;

    #pragma omp parallel for
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            for (int k = 0; k < c1; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }

    printf("\\nResult Matrix:\\n");
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++)
            printf("%d\\t", C[i][j]);
        printf("\\n");
    }
    return 0;
}
```

Compile and run

```
gcc -fopenmp matrix_multiplication.c -o matrix_multiplication
./matrix_multiplication
```

Example input

```
2 3
3 2
1 2 3
4 5 6
7 8
9 10
11 12
```

Expected result matrix

```
Result Matrix:
58      64
139     154
```

Explanation

• **Dimension condition:** columns of A must equal rows of B.

• **Three loops:** i selects a row of A, j selects a column of B, and k multiplies matching row/column elements and adds the products.

• **OpenMP:** `#pragma omp parallel for` distributes outer-loop iterations among threads.

• **Complexity:** O(r1 × c1 × c2) arithmetic operations.

2. CUDA Prime Number Finder (2 to 50)

Purpose: Each GPU thread checks one integer from 2 through 50 and records whether it is prime.

Source file: **prime_cuda.cu**

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void findPrimes(int *prime) {
    int num = threadIdx.x + 2;
    int flag = 1;

    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            flag = 0;
            break;
        }
    }
    prime[num] = flag;
}

int main(void) {
    const int N = 50;
    int prime[N + 1];
    int *d_prime;

    cudaMalloc((void**)&d_prime, (N + 1) * sizeof(int));
    cudaMemset(d_prime, 0, (N + 1) * sizeof(int));

    findPrimes<<<1, 49>>>(d_prime);
    cudaDeviceSynchronize();

    cudaMemcpy(prime, d_prime, (N + 1) * sizeof(int),
               cudaMemcpyDeviceToHost);

    printf("Prime Numbers:\\n");
    for (int i = 2; i <= N; i++)
        if (prime[i])
            printf("%d ", i);
    printf("\\n");

    cudaFree(d_prime);
    return 0;
}
```

Compile and run

```
nvcc prime_cuda.cu -o prime_cuda
./prime_cuda
```

Expected output

```
Prime Numbers:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Explanation

• **Kernel:** `__global__` marks a function that runs on the GPU.

• **Thread mapping:** `threadIdx.x + 2` maps thread IDs 0–48 to numbers 2–50.

• **Prime test:** if any divisor from 2 through num/2 divides the number evenly, it is not prime.

• **Memory:** `cudaMalloc` allocates GPU memory; `cudaMemcpy` copies results back to the CPU; `cudaFree` releases memory.

Requirements: NVIDIA CUDA Toolkit, a compatible NVIDIA GPU, and a working CUDA driver. Check CUDA calls for errors in production code.

3. CUDA Sum Reduction

Purpose: Calculate the sum of eight numbers using parallel pairwise additions on the GPU.

Source file: **sum_reduction.cu**

```
#include <stdio.h>
#include <cuda_runtime.h>

#define N 8

__global__ void reduceSum(int *input, int *output) {
    int tid = threadIdx.x;

    for (int stride = N / 2; stride > 0; stride /= 2) {
        if (tid < stride)
            input[tid] += input[tid + stride];

        __syncthreads();
    }

    if (tid == 0)
        *output = input[0];
}

int main(void) {
    int h_input[N] = {1, 2, 3, 4, 5, 6, 7, 8};
    int *d_input, *d_output;
    int result;

    cudaMalloc((void**)&d_input, N * sizeof(int));
    cudaMalloc((void**)&d_output, sizeof(int));

    cudaMemcpy(d_input, h_input, N * sizeof(int),
               cudaMemcpyHostToDevice);

    reduceSum<<<1, N>>>(d_input, d_output);
    cudaDeviceSynchronize();

    cudaMemcpy(&result, d_output, sizeof(int),
               cudaMemcpyDeviceToHost);

    printf("Sum = %d\\n", result);

    cudaFree(d_input);
    cudaFree(d_output);
    return 0;
}
```

Compile and run

```
nvcc sum_reduction.cu -o sum_reduction
./sum_reduction
```

Expected output

```
Sum = 36
```

Explanation

• **Input:** the host array contains 1, 2, 3, 4, 5, 6, 7, 8.

• **Reduction:** stride takes values 4, 2, and 1. Threads add pairs at each stage.

• **Synchronization:** `__syncthreads()` makes threads in the block wait between reduction stages.

• **Final result:** thread 0 writes the total to `output`, which is copied back to the CPU.

Note: This simple implementation assumes exactly N = 8 threads in one block and uses a fixed reduction size.

4. MPI Parallel Sorting

Purpose: Split an eight-element array across four MPI processes, sort each local pair, gather the pairs, and sort the complete array on rank 0.

Source file: **mpi_sort.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int compare(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[]) {
    int rank, size;
    int data[8] = {8, 3, 7, 1, 9, 2, 5, 4};
    int local[2];
    int gathered[8];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 4) {
        if (rank == 0)
            printf("Run this program with exactly 4 processes.\\n");
        MPI_Finalize();
        return 1;
    }

    MPI_Scatter(data, 2, MPI_INT,
                local, 2, MPI_INT, 0, MPI_COMM_WORLD);

    qsort(local, 2, sizeof(int), compare);

    MPI_Gather(local, 2, MPI_INT,
               gathered, 2, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        qsort(gathered, 8, sizeof(int), compare);
        printf("Sorted Array:\\n");
        for (int i = 0; i < 8; i++)
            printf("%d ", gathered[i]);
        printf("\\n");
    }

    MPI_Finalize();
    return 0;
}
```

Compile and run

```
mpicc mpi_sort.c -o mpi_sort
mpirun -np 4 ./mpi_sort
```

Expected output

```
Sorted Array:
1 2 3 4 5 7 8 9
```

Explanation

- **MPI_Init:** starts the MPI environment.
- **MPI_Comm_rank / MPI_Comm_size:** get the process ID and total number of processes.
- **MPI_Scatter:** distributes two integers to each of four processes.
- **qsort:** sorts each process's local pair.
- **MPI_Gather:** collects the sorted pairs on rank 0.
- **Final sort:** rank 0 sorts all eight values and prints the result.

Important: The local sorts happen across processes, but the final full-array qsort on rank 0 is sequential. This is a simple MPI sorting demonstration, not a fully parallel merge sort.

Quick Command Reference

Run these commands from the directory where each source file is saved.

Program	Compile	Run
OpenMP matrix multiplication	gcc -fopenmp matrix_multiplication.c -o matrix_multiplication	./matrix_multiplication
CUDA prime finder	nvcc prime_cuda.cu -o prime_cuda	./prime_cuda
CUDA sum reduction	nvcc sum_reduction.cu -o sum_reduction	./sum_reduction
MPI sorting	mpicc mpi_sort.c -o mpi_sort	mpirun -np 4 ./mpi_sort

Troubleshooting

• **nvcc: command not found** — install/configure the NVIDIA CUDA Toolkit.

• **mpicc: command not found** — install an MPI implementation, for example Open MPI or MPICH.

• **OpenMP compiler errors** — use GCC and include the `-fopenmp` flag.

• **MPI launch issues** — ensure the MPI runtime is installed and use the launch command appropriate to your environment.